

Day 14 — Resilient AI Integration + Fault-Tolerant Caching

Objective

Enhance the AI backend to handle real-world failures by implementing:

- Retry mechanism
- Circuit breaker
- Timeout handling
- Fallback responses
- Fault-tolerant Redis caching

Before Day 14

The system previously:

- Called AI provider directly
- Failed completely if:
 - AI API was down
 - Network failed
 - Redis cache failed

Result: **Unstable backend**

Goal: Build a production-grade resilient system where:

Failures do NOT break the system

Architecture Enhancement

Before: Controller → AiService → AI Provider	After: Controller ↓ AiService (Cache + Orchestration) ↓ AI Provider (Resilience Layer) ↓ External AI API
---	---

Key Concepts Implemented

1. Retry Mechanism

- Automatically retries failed requests.
- Why? Temporary failures (network glitches, rate limits) should not fail immediately.

2. Circuit Breaker

- Stops calling failing service after repeated failures.
- Why? Prevents system overload and cascading failures.

3. Timeout Handling

- Stops long-running requests.
- Why? Avoids hanging requests and resource exhaustion.

4. Fallback Response

- Gracefully returns a response when AI fails.
- Why? User experience should not break.

5. Fault-Tolerant Caching

- Ensures Redis failure does NOT crash system.
- Why? Cache is optional, not critical.

Implementation

1. Polly Resilience Policies

Infrastructure/Resilience/AIResiliencePolicy.cs

```
public static IAsyncPolicy<HttpStatusCode> GetRetryPolicy()
{
    return HttpPolicyExtensions
        .HandleTransientHttpError()
        .OrResult(msg => msg.StatusCode == HttpStatusCode.TooManyRequests)
        .WaitAndRetryAsync(
            3,
            retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAttempt))
        );
}

public static IAsyncPolicy<HttpStatusCode> GetCircuitBreakerPolicy()
{
    return HttpPolicyExtensions
        .HandleTransientHttpError()
        .CircuitBreakerAsync(3, TimeSpan.FromSeconds(30));
}
```

2. HttpClient Configuration

Program.cs

```
builder.Services.AddHttpClient("AIClient")
    .AddPolicyHandler(AIResiliencePolicy.GetRetryPolicy())
    .AddPolicyHandler(AIResiliencePolicy.GetCircuitBreakerPolicy())
    .AddPolicyHandler(Policy.TimeoutAsync<HttpStatusCode>(5));
```

3. Provider-Level Resilience + Fallback

OpenAIProvider.cs

```
try
{
    var response = await _httpClient.PostAsJsonAsync(url, requestBody);
}
```

```

response.EnsureSuccessStatusCode();

return await response.Content.ReadAsStringAsync();
}
catch (Exception ex)
{
    _logger.LogError(ex, "AI provider failed");
    return " ⚠️ AI service temporarily unavailable. Please retry";
}

```

4. Fault-Tolerant Caching

AIService.cs

Cache GET (Safe)

```

string? cachedResult = null;

try
{
    cachedResult = await _cache.GetAsync(cacheKey);
}
catch (Exception ex)
{
    _logger.LogError(ex, "Cache GET failed");
}

```

Cache SET (Safe + No fallback caching)

```

if (!aiResult.StartsWith(" ⚠️ "))
{
    try
    {
        await _cache.SetAsync(cacheKey, aiResult, TimeSpan.FromMinutes(10));
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Cache SET failed");
    }
}

```

5. Fallback Detection

```

IsFallback = aiResult.StartsWith(" ⚠️ ")

```

Testing

Test 1 — Normal Flow

Swagger Request: { "prompt": "hello" }	{ "output": "Processed Response: ...", "isFallback": false }
--	---

Logs:

Cache MISS

HTTP call → 200

Cache SET

Test 2 — Cache HIT

Repeated request:

Logs: Cache HIT

Performance:

Scenario	Time
Without cache	~1800 ms
With cache	~20–40 ms

Test 3 — Redis Failure

When Redis is down:

Logs:

Cache GET failed

Cache SET failed

- API still returns response
- No 500 error

Test 4 — AI Failure

When AI API fails:

```
{  
  "output": " ⚠️ AI service temporarily unavailable...",  
  "isFallback": true  
}
```

Final System Behavior

Normal Case: Cache MISS → AI → Cache SET → Response

Cached Case: Cache HIT → Response (NO AI CALL)

Redis Failure: Cache fails → AI still works → Response returned

AI Failure: AI fails → Fallback returned → System stable